Summarized System Architecture for citizen bridge

Objective: Build a Citizen Engagement System MVP to streamline complaint submission, categorization, routing, tracking, and response management, improving citizen satisfaction and government efficiency.

Core Components:

1. **Frontend**:
    - **Web App**: Responsive interface for citizens, admins, and agency reps (Next.ts).
2. **Backend**:
    - **API Layer**: RESTful APIs (Node.js with Express or Spring Boot) for handling requests.
    - **Complaint Service**: Manages complaint lifecycle (submission, categorization, routing).
    - **Notification Service**: Sends email, or in-app notifications .
    - **Authentication Service**: Handles user auth (JWT or OAuth2) with PostgreSQL.
3. **Databases**:
    - **PostgreSQL**: Stores user, role, and agency data for structured, relational needs.
    - **MongoDB**: Stores complaints, categories, responses, and notifications for flexible, document-based data.
4. **AI/ML (Optional)**:
    - AI-assisted categorization and routing using NLP (e.g., keyword-based or pre-trained models) using gemini.
5. **Infrastructure**:
    - Cloud-hosted (Render) with containerization (Docker) for scalability.
    - **File Storage**: S3 or equivalent for complaint attachments.
    - **Caching**: Redis for frequently accessed data (e.g., categories, complaint statuses).

Key Features:

- **Complaint Submission**: Citizens submit complaints via web/mobile with text, media, and location.
- **Categorization & Routing**: Automatic (keyword-based or ML) and manual categorization, with rule-based routing to agencies.
- **Status Tracking**: Citizens view real-time status updates and history.
- **Response Management**: Agencies respond, update statuses, and add internal notes.
- **Admin Interface**: Manage users, agencies, categories, and view analytics.
- **Notifications**: Real-time updates via email, SMS, or in-app messages.

Non-Functional Requirements:

- **Usability**: Intuitive UI/UX for all user types.

- **Scalability**: Handle thousands of complaints daily with horizontal scaling.
- **Extensibility**: Modular design for adding features like analytics dashboards or chatbots.
- **Security**: Encrypted data, role-based access control (RBAC), and audit logging.

Analysis

Stakeholders and Roles

1. **Citizens**:
    - Register, submit complaints, track status, and provide feedback.
    - Need simple, accessible interfaces (web/mobile).
2. **Government Admins**:
    - Review, categorize, and assign complaints to agencies.
    - Require analytics and reporting tools.
3. **Agency Representatives**:
    - Respond to assigned complaints and update statuses.
    - Need clear task management and communication tools.
4. **System Administrators**:
    - Manage users, agencies, categories, and system settings.
    - Require robust configuration and monitoring tools.

User Needs

- **Citizens**: Easy submission, transparent tracking, timely responses.
- **Admins**: Efficient complaint management, clear categorization rules.
- **Agency Reps**: Prioritized task lists, response templates.
- **Sys Admins**: Secure user management, system health monitoring.

Functional Requirements

1. **Complaint Submission**:
    - Input: Text, media, location, category selection.
    - Output: Unique reference number, confirmation notification.
2. **Categorization**:
    - Auto-categorization using keywords or ML.
    - Manual override by admins.
3. **Routing**:
    - Rule-based (e.g., location, category) or manual assignment to agencies.
4. **Tracking**:
    - Real-time status (Submitted, Under Review, Assigned, In Progress, Resolved, Closed).
    - Status history and audit trail.
5. **Response Management**:

    - Public responses and internal notes.
    - Resolution feedback from citizens.
6. **Admin Interface**:
    - Manage users, agencies, categories.
    - Basic analytics (e.g., complaints by category, resolution time).

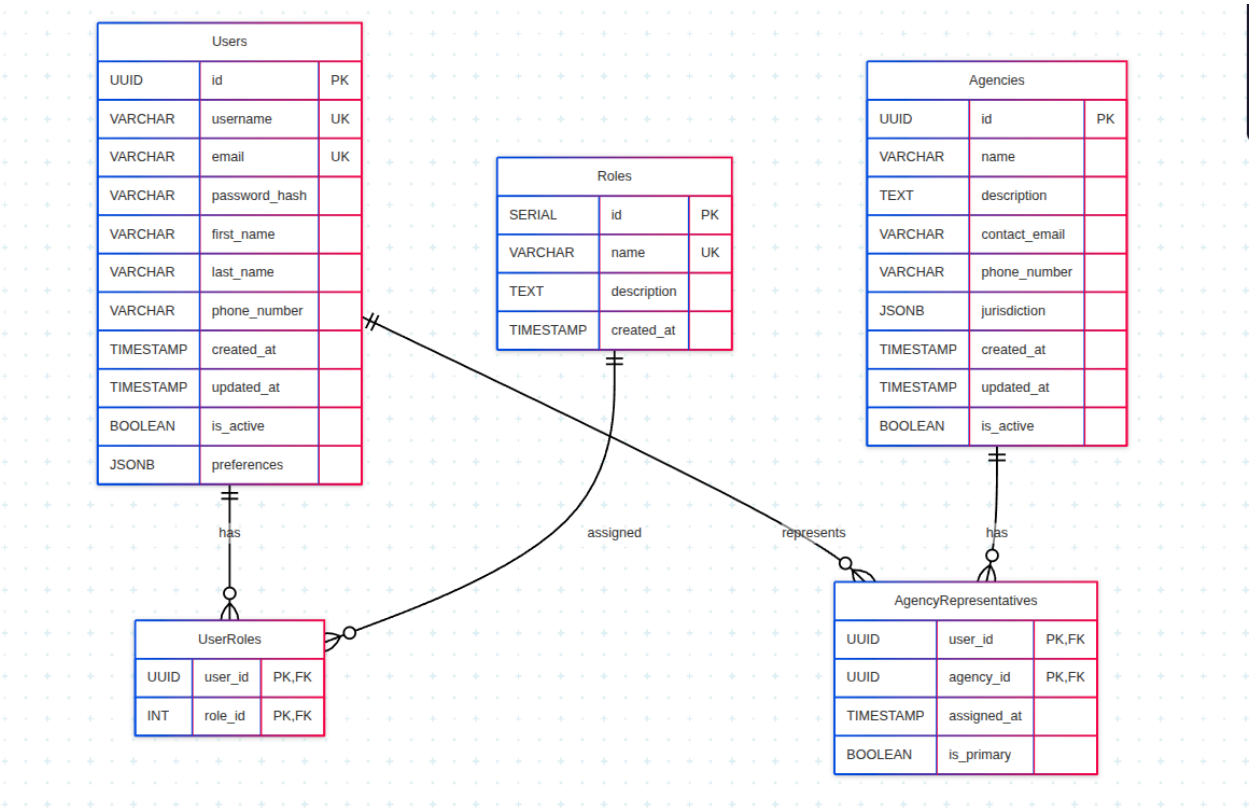
Technical Constraints

- **PostgreSQL**: Structured user data with strong consistency.
- **MongoDB**: Flexible complaint data with high write throughput.
- **Scalability**: Handle peak loads during public events or crises.
- **Interoperability**: APIs for future integration with other government systems.

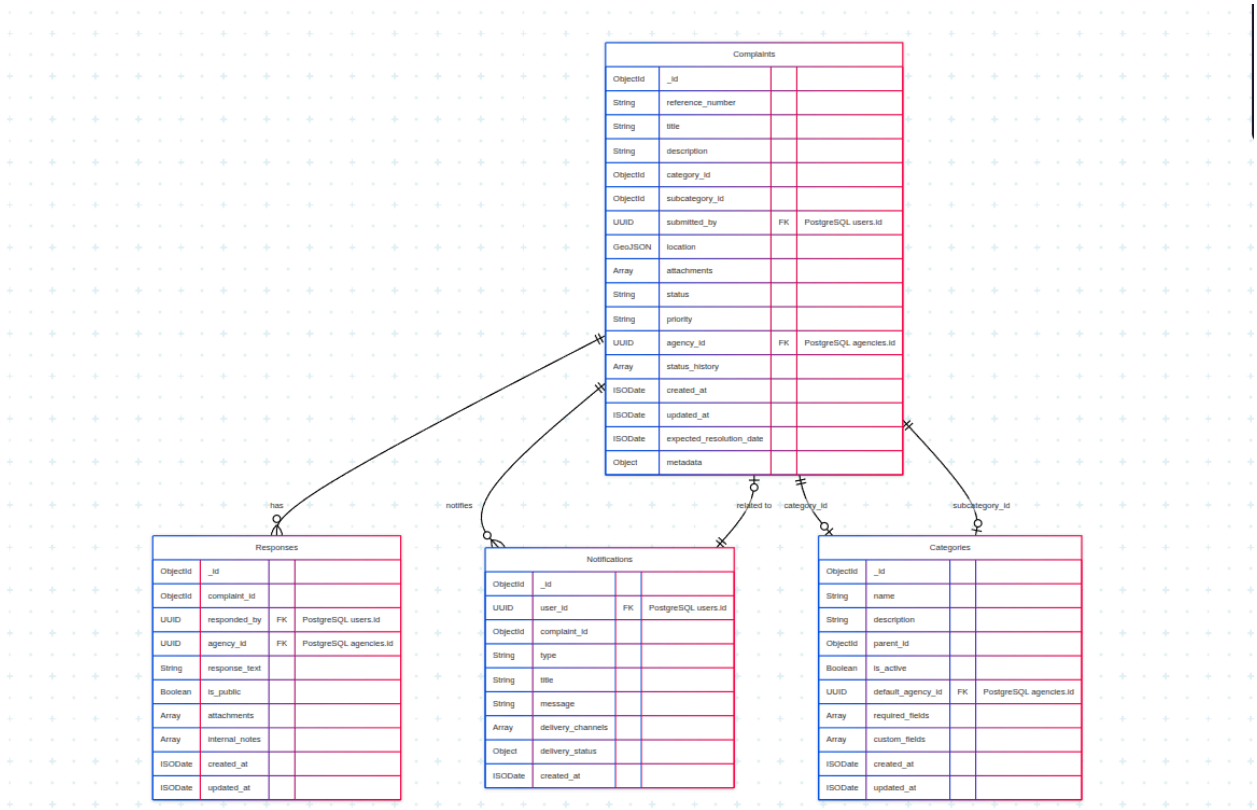
Database Design

PostgreSQL Schema (User Data)

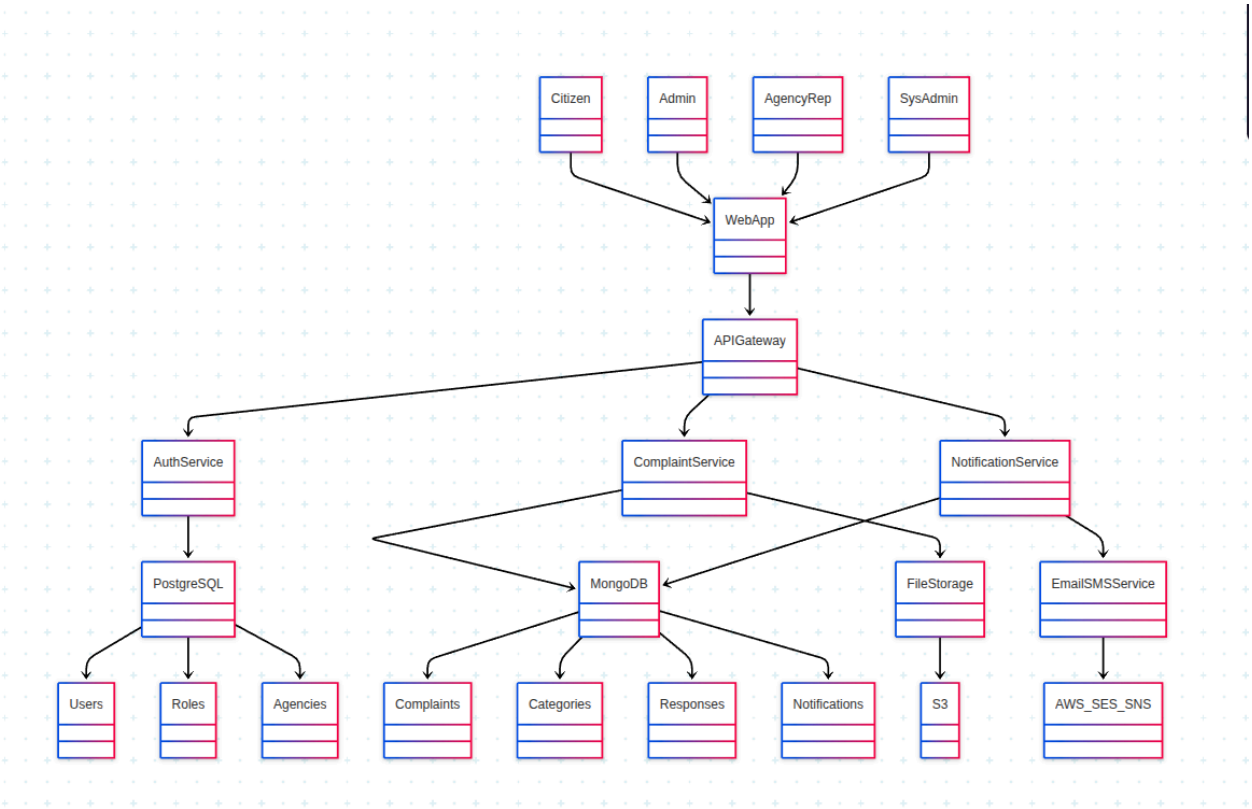
Stores structured, relational data for authentication and user management.



MongoDB Schema (Complaint Data)



System Architecture Diagram


Citizen
Admin
AgencyRep
SysAdmin
WebApp
APIGateway
AuthService
ComplaintService
NotificationService
PostgreSQL
MongoDB
FileStorage
EmailSMSService
Users
Roles
Agencies
Complaints
Categories
Responses
Notifications
S3
AWS_SES_SNS